

University of Wah
Department of Computer Science
BS Cyber Security
Blockchain Technology
Project Report



Title: BlockLog Guardian: Decentralized Tamper-Proof Authentication and Security Logging System using Ethereum (Geth) and IPFS

Submitted to: Ma'am Muniba Khan

Group Members:

Hassan Iftikhar (UW-23-CY-BS-002)

Muhammad Azfar Waqas (UW-23-CY-BS-013)

Ibrar Ul Hassan Shami (UW-23-CY-BS-018)

Section: BS-CYS-6N

Table of Contents

1. Introduction	1
1.1 Purpose.....	1
1.2 Project Evolution	1
1.3 System Architecture	2
2. Blockchain Implementation with Geth	3
2.1 Private Ethereum Network Setup.....	3
2.2 Geth Configuration	4
2.3 Consensus Mechanism.....	5
3. Smart Contract Development.....	6
3.1 SecurityLog Contract	6
3.2 ForensicEvidence Contract	7
3.3 Contract Deployment	7
4. Overall Description	8
4.1 System Overview	8
4.2 Manageability	8
4.3 User Classes and Characteristics	8
5. Threat Modeling.....	9
5.1 Threat Model Overview	9
5.2 Identified Threats	9
5.3 Security Relevance.....	10
6. Non-Functional Requirements	10
6.1 Performance Requirements	10
6.2 Security Requirements	11
6.3 Software Quality Attributes	11
7. Other Requirements	12
7.1 Hardware Interface.....	12
7.2 Communication Interface.....	12
8. System Execution and Deployment	12
8.1 Blockchain Node Setup.....	12
8.2 Smart Contract Deployment	13
8.3 Django Backend Setup.....	13
8.4 React Frontend Setup.....	14
8.5 System Access.....	14

9. Features and Demonstration	15
9.1 Key Features	15
9.2 Demonstration Flow.....	15
10. Conclusion	16
11. References	17

1. Introduction

1.1 Purpose

Modern organizations rely heavily on security audit logs to detect intrusions, investigate breaches, and demonstrate regulatory compliance. However, traditional logging systems store records in centralized databases that administrators, attackers, or malicious insiders can edit or delete after a breach, destroying the very evidence needed for forensic investigation.

BlockLog Guardian addresses this gap by combining enterprise-grade blockchain technology with decentralized storage. Unlike our initial proof-of-concept using a custom blockchain, this implementation leverages:

- **Ethereum Private Network with Geth** – A production-ready blockchain client providing robust consensus mechanisms and smart contract capabilities
- **IPFS (InterPlanetary File System)** – Decentralized storage for log records
- **Django Backend** – Enterprise-grade web framework with comprehensive security features
- **React Frontend** – Modern, responsive dashboard with real-time updates
- **Solidity Smart Contracts** – Immutable logic for security logging and forensic evidence management

Every authentication attempt, administrative action, and security event in the system is converted into a JSON record, uploaded to IPFS to obtain a unique Content Identifier (CID), and then permanently stored on the Ethereum blockchain through smart contracts. Any later modification to a stored log changes its CID and breaks the cryptographic verification, instantly revealing tampering.

1.2 Project Evolution

From Custom Blockchain to Geth-Powered Solution:

Our initial implementation used a custom SHA-256 blockchain with IPFS integration to demonstrate the core concepts of tamper-proof logging. However, to align with industry standards and provide a more robust, production-ready solution, we migrated to:

- **Geth (Go Ethereum)** – The official Ethereum client for building private networks
- **Clique Consensus** – Proof-of-Authority consensus for private/permissioned networks
- **Solidity Smart Contracts** – Formal, auditable logic for logging and evidence management
- **3-Node Architecture** – Distributed network with Miner, Replication, and Consensus/Audit nodes

This migration provides:

- Industry-standard blockchain infrastructure
- Smart contract programmability
- Multi-node consensus and redundancy
- No single point of failure
- Production-ready deployment capabilities

1.3 System Architecture

The project consists of six major components:

1. Ethereum Private Network (Geth)

- Three-node private Ethereum network with Clique consensus
- Node 1: Primary Miner and Signer
- Node 2: Replication and Verification Node
- Node 3: Consensus and Audit Node
- All nodes synchronized and maintaining the same blockchain state

2. Smart Contracts (Solidity)

- **SecurityLog Contract** – Manages immutable security event logging
 - Functions: addLog, getLog, getLogHistory, verifyLogIntegrity
 - Stores event type, description, timestamp, user address, metadata, and cryptographic hash
 - Implements owner-only access control for administrative functions
- **ForensicEvidence Contract** – Handles digital evidence preservation
 - Functions: addEvidence, getEvidence, verifyEvidence
 - Stores evidence ID, type, description, hash, collector, and chain of custody
 - Maintains verification status for forensic integrity

3. Authentication Module (Django)

- User registration and login with JWT authentication
- Session management and account protection
- OTP verification capability
- Role-based access control (User, Admin, Security Analyst)
- Account lock protection with failed login tracking

- Secure password hashing with bcrypt

4. Blockchain Integration Service (Web3.py)

- Connects Django backend to Geth nodes
- Interacts with smart contracts
- Handles transaction signing and submission
- Monitors blockchain status and events

5. IPFS Storage Module

- Decentralized storage of detailed log records
- Content-addressed storage with CID verification
- Automatic fallback to mock storage when IPFS daemon is unavailable

6. SOC Dashboard and Visualization Layer

- React-based frontend with real-time monitoring
- Blockchain status monitoring
- Smart contract interaction
- Security event timeline
- Analytics and visualization (Chart.js)
- Forensic incident reporting

2. Blockchain Implementation with Geth

2.1 Private Ethereum Network Setup

Network Configuration:

- **Network ID:** 2026
- **Consensus:** Clique (Proof-of-Authority)
- **Block Time:** 5 seconds
- **Gas Limit:** 8,000,000
- **Chain ID:** 2026

Node Architecture:

Node	Address	Port (HTTP)	Port (P2P)	Role
Node 1	0x4e0A...F55DbA	8545	30303	Primary Miner/Signer
Node 2	0x8A0e...36d113	8548	30304	Replication/Verification
Node 3	0xE35...D0Ff7	8550	30305	Consensus/Audit

Key Features:

- Distributed ledger storage with multi-node replication
- Clique consensus for fast, permissioned block production
- Immutable transactions with cryptographic verification
- No single point of failure
- Peer-to-peer synchronization
- Network-based integrity protection

2.2 Geth Configuration

Genesis Block Configuration:

json

```
{
  "config": {
    "chainId": 2026,
    "clique": {
      "period": 5,
      "epoch": 30000
    }
  },
  "difficulty": "1",
  "gasLimit": "8000000"
}
```

Node Startup Options:

- HTTP-RPC enabled for JSON-RPC API access
- WebSocket support for real-time subscriptions
- Mining enabled on Node 1
- Account unlocking with password file
- JWT authentication for authenticated RPC endpoints

2.3 Consensus Mechanism**Clique (Proof-of-Authority):**

- Chosen for private network deployment
- Faster block times compared to Proof-of-Work
- Trusted signers (Node 1 serves as the initial signer)
- In-turn and out-of-turn block production
- Automatic block sealing with configurable period

Advantages for Security Logging:

- Predictable block times for consistent logging
- No energy-intensive mining required
- Permissioned network ensures only trusted nodes participate
- Lower transaction costs (minimal gas requirements)

3. Smart Contract Development

3.1 SecurityLog Contract

Purpose: Provides immutable storage for security events

Key Functions:

Data Structure:

Function	Description	Access Control
addLog()	Records a new security event	Only contract owner
getLog()	Retrieves a specific log entry	Public view
getLogHistory()	Returns all log entries	Public view
verifyLogIntegrity()	Verifies log hash matches stored value	Public view
getTotalLogs()	Returns total number of logs	Public view

solidity

```
struct LogEntry {  
    uint256 timestamp;  
    string eventType;  
    string description;  
    string userAddress;  
    string metadata;  
    bytes32 hash;  
    bool verified;  
}
```

3.2 ForensicEvidence Contract

Purpose: Manages digital evidence for forensic investigations

Key Functions:

Function	Description	Access Control
addEvidence()	Adds new evidence record	Only contract owner
getEvidence()	Retrieves specific evidence	Public view
verifyEvidence()	Marks evidence as verified	Only contract owner
getEvidenceCount()	Returns total evidence count	Public view

Data Structure:

solidity

```
struct Evidence {  
    uint256 timestamp;  
    string evidenceId;  
    string evidenceType;  
    string description;  
    string hash;  
    string collector;  
    bool verified;  
}
```

3.3 Contract Deployment

Deployment Details:

- **Network:** Private Ethereum (Chain ID: 2026)
- **Deployer Address:** 0x4e0A20b9e3D7FaC87743f76dBA08b4f8CdF55DbA
- **SecurityLog Address:** 0xc00bE64D3f1049188a85cedCF07F909DA953590a
- **ForensicEvidence Address:** 0xd270bF3A111912910F405bAEBb9A09df494Ba2d0

Verification:

- Contracts compiled with Solidity 0.8.0
- ABI and bytecode stored in build directory
- Contract owners confirmed via on-chain queries

4. Overall Description

4.1 System Overview

BlockLog Guardian demonstrates how an enterprise-grade blockchain (Geth) combined with smart contracts and decentralized storage creates a tamper-evident audit trail. The platform:

1. Captures every authentication event and security incident
2. Stores detailed log records in IPFS
3. Records cryptographic references on the Ethereum blockchain
4. Provides a SOC dashboard with real-time monitoring
5. Enables forensic investigation through immutable evidence

4.2 Manageability

The project is designed for enterprise-grade manageability:

- **Modular Architecture:** Independent components for blockchain, IPFS, authentication, and presentation
- **Configuration Management:** Environment variables for all sensitive settings
- **Monitoring:** Prometheus-compatible metrics endpoints
- **Logging:** Structured logging with JSON format
- **Backup:** Smart contract addresses saved in configuration files

4.3 User Classes and Characteristics

Regular User:

- Registers and logs into the system
- Generates security events through authentication activity
- Can view profile and change password
- All activities logged immutably

System Administrator:

- Manages users and account lockouts
- Triggers security event simulations
- Has access to all SOC dashboard pages
- Administrative actions permanently recorded

Security Analyst / Forensic Investigator:

- Uses Blockchain Explorer for record review
- Validates chain integrity through verification engine
- Reviews incidents and generates forensic reports
- Traces events back to IPFS records

5. Threat Modeling

5.1 Threat Model Overview

The project evaluates security implications of centralized logging and demonstrates blockchain-backed mitigation strategies.

Primary Assets:

- Security audit logs (stored on blockchain via smart contracts)
- IPFS-stored event records
- User authentication credentials
- Smart contract state
- Administrative session privileges

5.2 Identified Threats

Threat	Description	Mitigation Strategy
Log Tampering	Modification/deletion of security logs	Smart contract immutability + IPFS CID verification
Brute Force Attack	Repeated login attempts	Account lockout after 5 failed attempts

Threat	Description	Mitigation Strategy
Unauthorized Access	Access to admin-only functions	Role-based access control with decorators
Privilege Escalation	Regular user gaining admin rights	Session-based role checks + audit logging
Account Takeover	Credential theft	Hashing + failed attempt logging + lockout
Chain Manipulation	Altering block data	Geth consensus validation + Clique signers
Smart Contract Exploit	Vulnerabilities in contract code	Owner-only functions + input validation

5.3 Security Relevance

This project demonstrates:

- **Decoupling:** Storage (IPFS/Blockchain) separated from verification
- **Immutability:** Smart contracts provide permanent, unalterable records
- **Forensic Readiness:** Every event traceable to original record
- **Cryptographic Verification:** Merkle-proof style integrity checks
- **Trust Minimization:** Trust moved from administrators to code

6. Non-Functional Requirements

6.1 Performance Requirements

- **Block Confirmation:** New blocks mined every 5 seconds (Clique consensus)
- **Event Recording:** Security events confirmed within one block (5-10 seconds)
- **Dashboard Updates:** Real-time updates via WebSocket connections
- **API Response:** REST endpoints respond within 200ms
- **Chart Rendering:** Responsive rendering even with 1000+ events

6.2 Security Requirements

- **Password Hashing:** bcrypt with salt (production) / SHA-256 (demonstration)
- **Account Lockout:** After 5 consecutive failed login attempts
- **Session Management:** JWT tokens with 60-minute expiration
- **CORS Protection:** Restricted to specific origins in production
- **Smart Contract Access:** Owner-only functions protected by modifiers
- **RPC Security:** JWT authentication for authenticated endpoints

6.3 Software Quality Attributes

Reliability:

- Geth nodes automatically reconnect on failure
- IPFS falls back to mock storage when daemon unavailable
- Smart contracts use require statements for validation

Maintainability:

- Separation of concerns (blockchain, storage, web, UI)
- Environment-based configuration
- Comprehensive error handling and logging

Scalability:

- Ethereum network can add additional nodes
- IPFS storage scales horizontally
- Django application can be containerized and load-balanced

Usability:

- Unified SOC dashboard interface
- Visual indicators for system status
- Interactive blockchain explorer

7. Other Requirements

7.1 Hardware Interface

- Development: Personal Computer (Intel Core i3+, 8GB RAM)
- Production: Server with 4GB+ RAM, SSD storage
- Optional: Dedicated machine for Geth nodes

7.2 Communication Interface

- **HTTP/HTTPS:** Django REST API endpoints
- **WebSocket:** Real-time blockchain monitoring
- **JSON-RPC:** Geth node communication via Web3.py
- **IPFS API:** Kubo HTTP API (<http://127.0.0.1:5001>)
- **GraphQL:** Optional for advanced queries

8. System Execution and Deployment

8.1 Blockchain Node Setup

Step 1: Install Geth

text

```
# Download Geth v1.13.15
```

```
curl -L -o geth.zip https://gethstore.blob.core.windows.net/builds/geth-windows-amd64-1.13.15-75d67b6c.zip
```

```
unzip geth.zip -d geth/
```

Step 2: Initialize Genesis Block

text

```
geth --datadir blockchain/data/node1 init blockchain/genesis.json
```

```
geth --datadir blockchain/data/node2 init blockchain/genesis.json
```

```
geth --datadir blockchain/data/node3 init blockchain/genesis.json
```

Step 3: Start Nodes

bash

```
# Terminal 1 - Node 1 (Miner)
```

```
./scripts/start_node1.sh
```

```
# Terminal 2 - Node 2 (Replication)
```

```
./scripts/start_node2.sh
```

```
# Terminal 3 - Node 3 (Audit)
```

```
./scripts/start_node3.sh
```

```
# Terminal 4 - Connect Nodes
```

```
./scripts/connect_nodes.sh
```

8.2 Smart Contract Deployment

Step 4: Compile Contracts

```
bash
```

```
python scripts/compile_contracts.py
```

Step 5: Deploy Contracts

```
bash
```

```
python scripts/deploy_simple.py
```

Step 6: Verify Deployment

```
bash
```

```
python scripts/test_contracts.py
```

8.3 Django Backend Setup

Step 7: Install Dependencies

```
bash
```

```
pip install -r requirements.txt
```

Step 8: Configure Environment

```
bash
```

```
# Update .env with contract addresses
```

```
SECURITY_LOG_CONTRACT=0xc00bE64D3f1049188a85cedCF07F909DA953590a
```

```
FORENSIC_EVIDENCE_CONTRACT=0xd270bF3A111912910F405bAEBb9A09df494Ba  
2d0
```


Step 9: Run Migrations

bash

python manage.py makemigrations

python manage.py migrate

python manage.py createsuperuser

Step 10: Start Django Server

bash

python manage.py runserver

8.4 React Frontend Setup

Step 11: Install Dependencies

bash

cd frontend

npm install

Step 12: Start Development Server

bash

npm start

8.5 System Access

- **Django Admin:** <http://localhost:8000/admin/>
- **SOC Dashboard:** <http://localhost:3000/>
- **Blockchain Status:** <http://localhost:8000/api/blockchain/status/>
- **Geth Node 1 RPC:** <http://localhost:8545>

9. Features and Demonstration

9.1 Key Features

Feature	Description	Technology
Immutable Logging	Every security event permanently stored	Ethereum + Smart Contracts
Forensic Evidence	Digital evidence with chain of custody	ForensicEvidence Contract
Real-time Monitoring	Live blockchain status and events	WebSocket + React
Tamper Detection	Cryptographic verification of records	SHA-256 + Smart Contract
Decentralized Storage	Detailed logs stored on IPFS	IPFS + CID
Multi-Node Redundancy	3-node network with failover	Geth Clique
Authentication	JWT-based user authentication	Django + JWT
Analytics	Visual insights with Chart.js	React + Chart.js
Forensic Reporting	Comprehensive incident reports	Django + PDF Generation

9.2 Demonstration Flow

1. **Login** as Administrator (admin / Admin@1234)
2. **View Dashboard** – Confirm blockchain status and node connectivity
3. **Trigger Events** – Generate authentication attempts, admin actions
4. **Explore Blockchain** – Inspect blocks in the explorer
5. **Verify Integrity** – Run chain verification to confirm immutability

6. **Tamper Demonstration** – Modify a record and watch detection
7. **Forensic Investigation** – Generate forensic reports from evidence

10. Conclusion

This project successfully demonstrates the integration of enterprise-grade blockchain technology (Geth) with decentralized storage (IPFS) to create a tamper-evident security audit logging platform. By migrating from a custom blockchain to an Ethereum private network, we have achieved:

- **Production-Ready Infrastructure:** Geth provides battle-tested consensus mechanisms and network protocols
- **Smart Contract Programmability:** Solidity enables complex logic for logging and evidence management
- **Multi-Node Resilience:** Three-node architecture with no single point of failure
- **Industry Standardization:** Compatibility with Ethereum tooling and libraries

Key Achievements:

1. **Immutable Audit Trail:** Security events permanently recorded on blockchain
2. **Tamper Detection:** Cryptographic verification detects any modification
3. **Forensic Capability:** Chain of custody for digital evidence
4. **Real-time Monitoring:** SOC dashboard with live updates
5. **Scalable Architecture:** Designed for production deployment

Future Enhancements:

- Deploy on public testnet (Sepolia/Goerli)
- Implement multi-signature administrative controls
- Add advanced analytics and ML-based threat detection
- Integrate with enterprise SIEM platforms (Splunk, ELK)
- Deploy on cloud infrastructure with Kubernetes
- Add mobile application for remote monitoring

The architecture is designed for extension toward smart-contract-based access control, integration with enterprise security platforms, and deployment in production environments requiring cryptographically verifiable audit trails.

11. References

- [1] J. D. Morillo Reina and T. J. Mateo Sanguino, "Decentralized and Secure Blockchain Solution for Tamper-Proof Logging Events," *Future Internet*, vol. 17, no. 3, p. 108, 2025.
- [2] "LogStamping: A Blockchain-Based Log Auditing Approach for Large-Scale Systems," arXiv preprint, 2025.
- [3] "Towards Blockchain-Driven, Secure and Transparent Audit Logs," ResearchGate, 2025.
- [4] "BCALS: Blockchain-Based Secure Log Management System for Cloud Computing," ResearchGate, 2025.
- [5] Ethereum Foundation, "Go Ethereum (Geth) Documentation," [Online]. Available: <https://geth.ethereum.org/>
- [6] Protocol Labs, "IPFS Documentation," [Online]. Available: <https://docs.ipfs.tech/>
- [7] Pallets Project, "Django Documentation," [Online]. Available: <https://docs.djangoproject.com/>
- [8] Web3.py Documentation, [Online]. Available: <https://web3py.readthedocs.io/>
- [9] National Institute of Standards and Technology (NIST), "Secure Hash Standard (SHS)," FIPS PUB 180-4.
- [10] React Team, "React Documentation," [Online]. Available: <https://react.dev/>
- [11] Chart.js Contributors, "Chart.js Documentation," [Online]. Available: <https://www.chartjs.org/docs/latest/>
- [12] Bootstrap Team, "Bootstrap 5 Documentation," [Online]. Available: <https://getbootstrap.com/docs/5.3/>